

# SpringBoot3 教程

讲师：燕涛

## 准备开发环境

### JDK17

Java 17 在 2021 年 9 月 14 日正式发布，是一个长期支持（LTS）版本。

Java 17 将是继 Java 8 以来最重要的长期支持（LTS）版本，是 Java 社区八年努力的成果。

Spring 6.x 和 Spring Boot 3.x 最低支持的就是 Java 17。

### 安装前准备

下载 JDK17 安装包。

推荐从 Oracle 官网或 OpenJDK（免费开源）下载：

Oracle JDK17 : <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>

OpenJDK17（推荐，无版权限制）：<https://jdk.java.net/17/>

注意根据系统选择对应版本（Windows-x64、macOS-aarch64/amd64、Linux-x64）。

### Windows 系统安装步骤

#### 1. 执行安装程序

双击下载的.exe 安装包（如 jdk-17\_windows-x64\_bin.exe），点击「下一步」。

选择安装路径（建议默认路径：C:\Program Files\Java\jdk-17.x.x，或自定义路径，路径中不要包含中文、空格）。

等待安装完成，点击「关闭」。

#### 2. 配置环境变量

右键「此电脑」→「属性」→「高级系统设置」→「环境变量」。

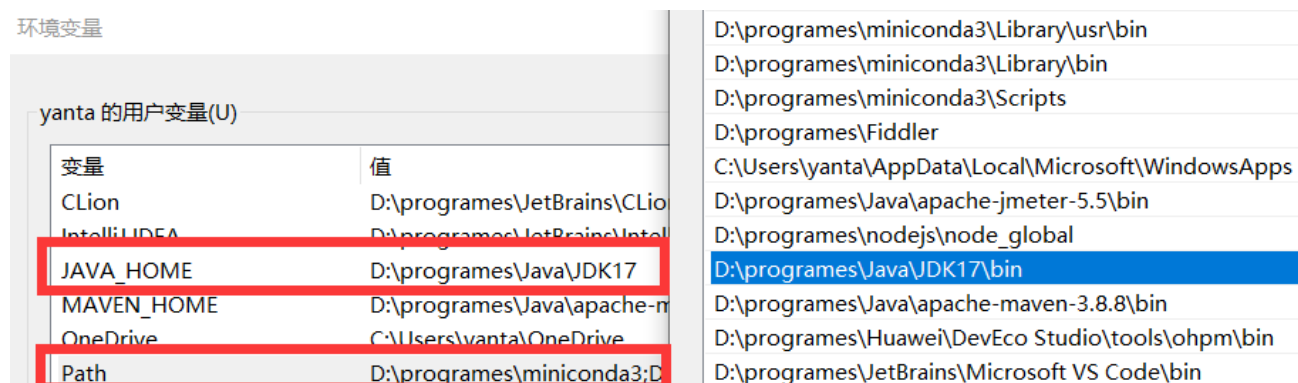
新建系统变量：

变量名：JAVA\_HOME

变量值：JDK 安装路径（如 C:\Program Files\Java\jdk-17.0.9）。

编辑系统变量 Path:

点击「新建」, 添加%JAVA\_HOME%\bin。



确保该条目在 Path 列表靠前位置, 避免和其他 JDK 版本冲突。

### 3. 验证安装

按下 Win+R, 输入 CMD 打开命令提示符。

执行以下命令, 若输出版本信息则安装成功:

Java, Javac, Java -version

```
C:\Users\yanta>java -version
java version "17.0.10" 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 17.0.10+11-LTS-240)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.10+11-LTS-240, mixed mode, sharing)
```

### ➤ 教学补充说明

JDK vs JRE vs JVM:

JDK 包含 JRE (运行环境) 和开发工具 (javac、java 等), JRE 包含 JVM (虚拟机), 简单类比: JDK 是 “开发工具箱”, JRE 是 “运行环境”, JVM 是 “跨平台核心”。

**环境变量作用:**

JAVA\_HOME: 告诉系统 JDK 的安装位置, 很多框架 (如 Maven、Tomcat) 会依赖该变量。

Path: 让系统在任意目录下都能执行 java、javac 等命令。

**多版本 JDK 切换:**

Windows: 修改 JAVA\_HOME 的路径为对应版本即可。

macOS/Linux: 使用 update-alternatives 命令配置。

## IntelliJ IDEA

IntelliJ IDEA 是一款广受欢迎的集成开发环境 (IDE)，专为 Java 编程语言设计，但也支持其他语言如 Kotlin、Scala 和 Groovy。由 JetBrains 公司开发，该公司以其严谨的东欧程序员团队而闻名。IDEA 被认为是业界最佳的 Java 开发工具之一。

### 安装与初始配置

#### ➤ 下载安装包

官网下载：<https://www.jetbrains.com/idea/download/>，选择版本：  
建议使用 2025.3 及以后版本，旗舰版与社区版合二为一。

#### ➤ windows 安装步骤

1. 双击 .exe，选择安装路径（建议非系统盘，无中文 / 空格）；
2. 勾选 “创建桌面快捷方式” “添加到 PATH” “关联 .java 文件”；
3. 安装完成后启动，首次运行需完成初始配置。

#### ➤ macos 安装步骤

1. 双击 .dmg，将 IDEA 拖拽到 Applications 文件夹；
2. 启动台打开，首次运行需在 “系统设置 - 隐私与安全性” 点击 “仍要打开”。

#### ➤ 核心配置

1. 导入配置：选择 “Do not import settings”（首次使用）。
2. 主题选择：Darcula（深色，护眼）或 Light（浅色）。
3. JDK 与依赖管理

全局 JDK 配置：File→Project Structure→Platform Settings→SDKs，添加 / 删除 JDK 版本，确保所有项目默认使用 JDK17。

4. Maven 配置（Java 项目常用构建工具）：

打开 “Settings” →Build, Execution, Deployment→Build Tools→Maven。

设置 Maven home directory（本地 Maven 路径）、User settings file (settings.xml，配置镜像加速下载依赖)。

## Maven

### Maven 核心概念

#### ➤ 什么是 Maven

Apache 旗下的项目管理和构建工具，核心解决 3 个问题：

**依赖管理：**自动下载、管理项目所需的 jar 包（如 Spring、MySQL 驱动），告别手动复制 jar 包。

**统一构建：**通过简单命令（`mvn clean package`）完成编译、测试、打包等流程。

**项目结构标准化：**规定统一的项目目录结构，让团队开发更规范。

#### ➤ 核心术语

POM.xml：Maven 的核心配置文件，包含项目信息、依赖、构建规则等。

坐标（GAV）：唯一标识一个依赖包，如同“快递地址”：

**GroupId：**组织 / 公司标识（如 `org.springframework`）

**ArtifactId：**项目 / 模块名称（如 `spring-core`）

**Version：**版本号（如 `5.3.25`）

仓库：存放 jar 包的地方：

本地仓库：本地电脑的 jar 包缓存（默认路径：`WindowsC:\Users\用户名\.m2\repository`，`macOS/Linux~/.m2/repository`）。

中央仓库：Maven 官方的远程仓库（`https://repo.maven.apache.org/maven2/`）。

镜像仓库：中央仓库的国内镜像（如阿里云），解决下载慢的问题。

重要网址：<https://mvnrepository.com/>

### Maven 安装与配置

#### 1. 安装前提

确保已安装 JDK17 并配置好 JAVA\_HOME 环境变量（验证：`java -version`）。

#### 2. 下载安装包

官网下载：`https://maven.apache.org/download.cgi`，选择二进制压缩包（`apache-maven-3.9.x-bin.zip/.tar.gz`）。

#### 3. 解压与环境变量配置

1) 解压到非中文 / 无空格路径 (如 D:\apache-maven-3.9.6);

2) 配置环境变量:

a) 新建系统变量 MAVEN\_HOME, 值为解压路径;

b) 编辑 Path, 添加%MAVEN\_HOME%\bin;

3) 验证: CMD 中执行 mvn -v。

#### 4. 关键配置

修改 Maven 的核心配置文件 settings.xml (路径: MAVEN\_HOME/conf/settings.xml), 建议复制到本地仓库目录 (~/.m2/settings.xml), 避免修改全局配置。

(1) 配置阿里云镜像 (解决下载慢)

在<mirrors>标签内添加:

```
<mirror>
  <id>aliyunmaven</id>
  <name>阿里云中央仓库</name>
  <url>https://maven.aliyun.com/repository/public</url>
  <mirrorOf>central</mirrorOf> <!-- 替代中央仓库 -->
</mirror>
```

(2) 自定义本地仓库路径 (可选)

在<settings>根标签内添加:

```
<localRepository>D:\maven-repository</localRepository>
```

#### 5. 验证配置

执行 mvn -v, 输出如下信息即安装成功:

```
C:\Users\yanta>mvn -v
Apache Maven 3.8.8 (4c87b05d9aedce574290d1acc98575ed5eb6cd39)
Maven home: D:\programes\Java\apache-maven-3.8.8
Java version: 17.0.10, vendor: Oracle Corporation, runtime: D:\programes\Java\JDK17
Default locale: zh_CN, platform encoding: GBK
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
```

#### Maven 核心操作

##### 1. 标准化项目结构

先让学生记住 Maven 的默认目录结构, 这是规范的核心:

项目根目录

```

├─ src
│   ├─ main          # 主程序代码
│   │   ├─ java      # Java源码（包+类）
│   │   └─ resources # 配置文件（如application.properties）
│   └─ test          # 测试代码
│       ├─ java      # 测试类（如JUnit测试）
│       └─ resources # 测试配置文件
└─ pom.xml           # Maven核心配置文件
  
```

## 快速创建 Maven 项目

### ➤ 方式 1：命令行创建（理解底层）

# 原型创建 Java 项目（archetype 为原型模板）

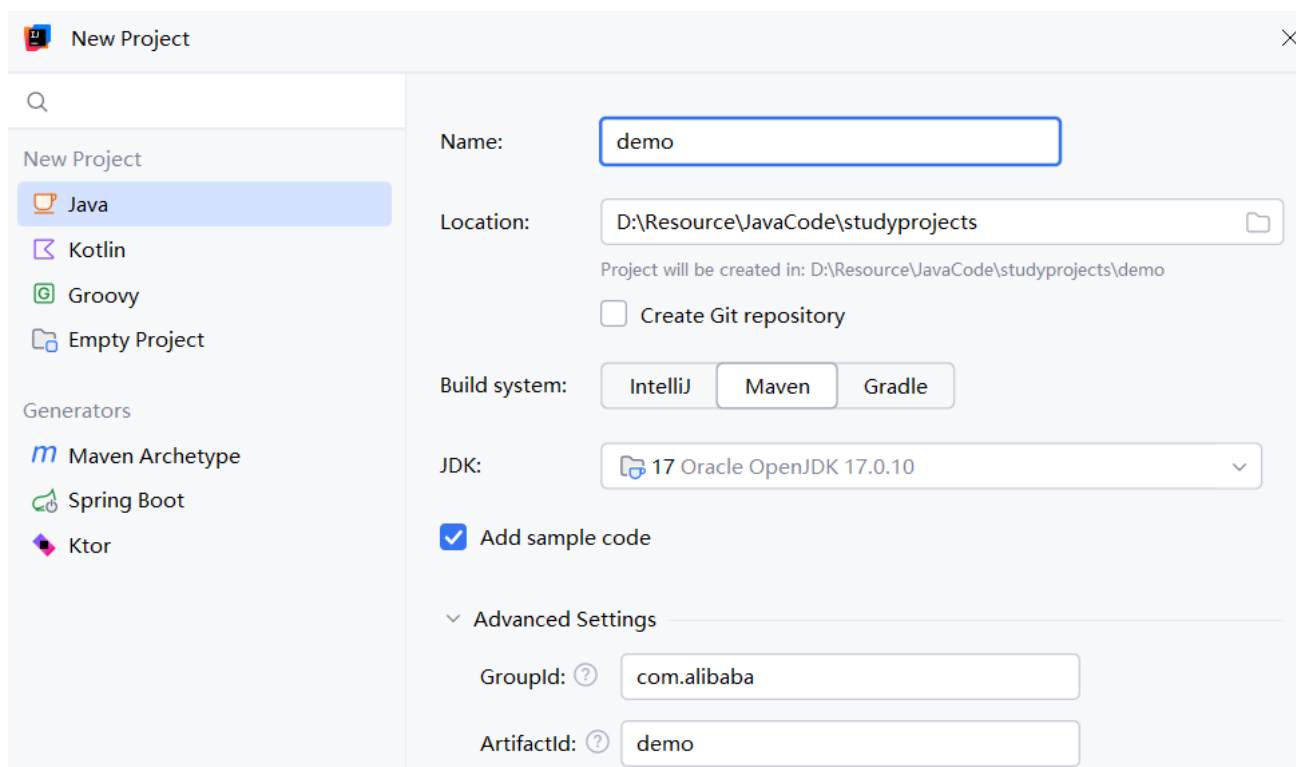
```

mvn archetype:generate -DgroupId=com.example -DartifactId=java-demo -Dversion=1.0.0 -
Dpackage=com.example -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
  
```

参数说明：

- `groupId`：组织标识（如学校/公司包名）
- `artifactId`：项目名
- `version`：版本号
- `package`：基础包名

### ➤ 方式 2：IDEA 创建



### ➤ POM.xml 核心配置

以一个简单的 Java 项目为例，讲解 POM.xml 的关键配置：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <!-- 模型版本，固定为4.0.0 -->
  <modelVersion>4.0.0</modelVersion>

  <!-- 项目坐标（GAV） -->
  <groupId>com.example</groupId>
  <artifactId>java-demo</artifactId>
  <version>1.0.0</version>

  <!-- 打包方式：jar（Java项目）、war（Web项目）、pom（父项目） -->
  <packaging>jar</packaging>

  <!-- 项目属性（可复用） -->
  <properties>
    <maven.compiler.source>17</maven.compiler.source> <!-- 编译JDK版本 -->
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> <!-- 编码 -->
  </properties>
```

```

<!-- 依赖管理：核心！自动下载jar包 -->
<dependencies>
  <!-- JUnit测试依赖 -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope> <!-- 作用域：仅测试环境生效 -->
  </dependency>
  <!-- MySQL驱动依赖 -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
  </dependency>
</dependencies>
</project>

```

关键说明：

- <scope>：依赖作用域，常用值：
- test：仅测试代码生效（如 JUnit）；
- compile：默认值，主程序和测试都生效；
- provided：运行时由容器提供（如 Servlet API）。

## 核心 Maven 命令

在项目根目录（pom.xml 所在目录）执行，建议结合 IDEA 的 Maven 面板讲解：

| 命令                             | 作用                                |
|--------------------------------|-----------------------------------|
| <code>mvn clean</code>         | 清理项目，删除target目录（编译/打包产物）。         |
| <code>mvn compile</code>       | 编译主程序代码，生成class文件到target/classes。 |
| <code>mvn test</code>          | 编译并运行测试代码，生成测试报告。                 |
| <code>mvn package</code>       | 打包项目（生成jar/war包到target目录）。        |
| <code>mvn install</code>       | 打包并将jar包安装到本地仓库，供其他项目依赖。          |
| <code>mvn clean package</code> | 组合命令：先清理，再打包（最常用）。                |

## ➤ IDEA 中使用 Maven

1. 打开 IDEA 的 Maven 面板（右侧边栏）：

- 展开项目→Lifecycle：双击对应命令（如 clean、package）即可执行。
- Dependencies：查看已添加的依赖，可右键添加/删除依赖。

2. 依赖自动导入：IDEA 默认开启 “Auto-Import”，修改 pom.xml 后自动下载依赖。

## 常见问题与排错

### 1. 依赖下载失败：

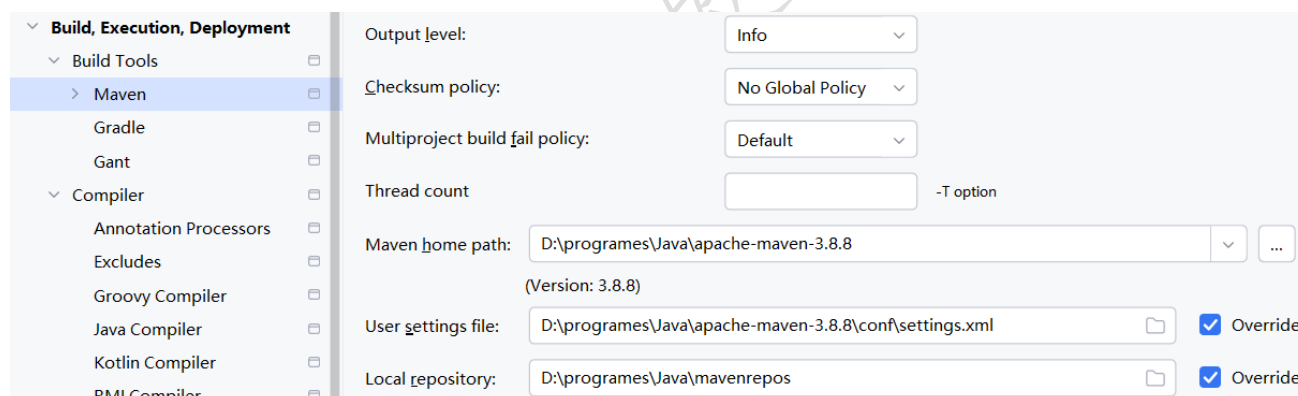
- 原因：网络问题、镜像配置错误、依赖坐标错误、本地仓库写保护。
- 解决：检查阿里云镜像配置；删除本地仓库中失败的依赖目录，重新执行`mvn clean compile`；验证 GAV 坐标（可在 <https://mvnrepository.com/> 查询）。

### 2. 编译报错 “无效的目标发行版：17”：

- 原因：Maven 编译版本与 JDK 版本不匹配。
- 解决：在 pom.xml 的`<properties>`中配置`maven.compiler.source`和`target`为 17，或在 IDEA 中设置 Project SDK 为 17。

### 3. 本地仓库路径修改不生效：

- 原因：修改了全局 settings.xml 但未生效，或 IDEA 读取了默认路径。
- 解决：在 IDEA 的 Maven 设置中指定 User settings file 路径。



## Spring 框架

官网: <https://spring.io/>

Spring 是一个支持快速开发 Java EE 应用程序的框架。它提供了一系列底层容器和基础设施，并可以和大量常用的开源框架无缝集成，可以说是开发 Java EE 应用程序的必备。

Spring 最早是由 Rod Johnson 在他的《Expert One-on-One J2EE Development without EJB》一书中提出的用来取代 EJB 的轻量级框架。随后其专心开发这个基础框架，并起名为 Spring Framework。

随着 Spring 越来越受欢迎，在 Spring Framework 基础上，又诞生了 Spring Boot、Spring Cloud、Spring Data、Spring Security 等一系列基于 Spring Framework 的项目。本章我们只介绍 Spring Framework，即最核心的 Spring 框架。

### ➤ 什么是 Spring?

Spring makes programming Java quicker, easier, and safer for everybody. Spring's focus on speed, simplicity, and productivity has made it the world's most popular Java framework.

Spring 使 Java 编程更快、更容易、更安全。Spring 对速度、简单性和生产力的关注使它成为世界上最流行的 Java 框架。

### ➤ Spring Framework 主要包括几个模块:

支持 IoC 和 AOP 的容器;

支持 JDBC 和 ORM 的数据访问模块;

支持声明式事务的模块;

支持基于 Servlet 的 MVC 开发;

支持基于 Reactive 的 Web 开发;

以及集成 JMS、Java Mail、JMX、缓存等其他模块。

## 教学目标

理解 Spring IoC 容器的核心思想。

掌握纯 Java Config 方式配置 Spring 容器。

熟练使用依赖注入（DI）、组件扫描、AOP 等核心功能。

能基于 Java Config 开发简单的 Spring 应用。

### ➤ 环境要求

JDK 17、Maven 3.8.8+（依赖管理）、IDE：IntelliJ IDEA 2025.3

Spring Context 6.x

## Spring 核心思想

IoC（Inverse of Control 控制反转）：将原本在程序中手动创建对象（new）的控制权，交由 Spring 框架管理，简单说，就是创建对象控制权被反转到了 spring 框架。

DI（Dependency Injection 依赖注入）：在 spring 框架创建 Bean 对象时，动态的将依赖对象注入到 Bean 组件中（如构造器注入、Setter 注入）。

Java Config：通过 Java 类 + 注解替代 XML 文件，实现 Spring 配置，核心注解：@Configuration、@Bean、@ComponentScan、@Component、@Autowired 等。

## 快速入门

第一步：使用 Idea 创建一个 Maven 工程，并引入以下依赖。

```
<dependencies>
  <!-- Spring核心上下文 -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.7</version>
  </dependency>
  <!-- JUnit（单元测试） -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

第二步：创建 Person 类与 Hello 类

```
public class Hello { no usages
    public void sayHello(){ no usages
        System.out.println("Hello Spring...");
    }
}

public class Person { 5 usages
    private String name; 3 usages
    private int age; 3 usages
    private String address; 3 usages
}
```

第三步：创建单元测试方法。说明，这是用传统的方式创建并使用类。

```
@Test
public void test01(){
    Hello hello = new Hello();
    hello.sayHello();

    Person person = new Person();
    person.setName("李雷");
    person.setAge(21);
    person.setAddress("湖南长沙");
    System.out.println(person);
}
```

第四步：加入 Spring 对应的注解。

```
@Configuration //标记类为 Spring 配置类，替代 XML 的<beans>标签
@ComponentScan("com.edu.springedu") //配置容器自动扫描指定包下的组件
public class SpringConfig {

    @Bean //标记方法返回值为 Spring 容器管理的 Bean的实例
    public Hello getHello(){
        return new Hello();
    }
}

@Component //标记普通类为 Spring 组件（衍生注解：@Service、@Repository、@Controller）
public class Person {

    @Value("韩梅梅") //属性依赖注入
    private String name;

    @Value("18")
    private int age;

    @Value("中国深圳")
    private String address;
}
```

```

public class Hello { 8 usages

    @Autowired //自动注入依赖的 Bean
    private Person person;

    public void sayHello(){ 2 usages
        System.out.println("Hello," + person);
    }
}

@Test
public void test02(){
    ApplicationContext ctx = new AnnotationConfigApplicationContext(SpringConfig.class);
    Hello hello = ctx.getBean(Hello.class);
    hello.sayHello();
    Person person = ctx.getBean(Person.class);
    System.out.println(person);
}

```

第五步：完成测试。对比二种方式的区别，理解 IOC 与 DI。

✓ 1 test passed 1 test total, 631 ms

```

D:\programes\Java\JDK17\bin\java.exe ...
Hello,Person{name='韩梅梅', age=18, address='中国深圳'}
Person{name='韩梅梅', age=18, address='中国深圳'}

```

## ➤ 本节总结

@Configuration: 标记类为 Spring 配置类，替代 XML 的<beans>标签。

@Bean: 标记方法返回值为 Spring 容器管理的 Bean，替代 XML 的<bean>标签。

AnnotationConfigApplicationContext: 基于注解的 Spring 容器，专门适配 Java Config。

@Bean 的 name 属性: 可自定义 Bean 的 id，如@Bean(name = "myBean")。

@Component: 标记普通类为 Spring 组件(衍生注解: @Service、@Repository、@Controller)。

@ComponentScan: 配置容器自动扫描指定包下的组件，默认扫描当前包及其子包，可通过 basePackages 指定多个包。

@Autowired: 自动注入依赖的 Bean，注入规则: 先按类型匹配，类型唯一则直接注入；类型不唯一则按名称匹配。

@Value: 注入普通类型属性。

## ➤ Bean 作用域 (@Scope)

默认 Bean 为单例 (singleton)，可通过@Scope 修改：

```
// 原型作用域：每次获取Bean都创建新对象
@Bean // 标记方法返回值为 Spring 容器管理的 Bean的实例
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public Hello getHello(){
    return new Hello();
}
```

## AOP 配置

AOP 是 Aspect Oriented Programming，即面向切面编程。

我们先回顾一下 OOP：Object Oriented Programming，OOP 作为面向对象编程的模式，获得了巨大的成功，OOP 的主要功能是数据封装、继承和多态。

而 AOP 是一种新的编程方式，它和 OOP 不同，OOP 把系统看作多个对象的交互，AOP 把系统分解为不同的关注点，或者称之为切面 (Aspect)。

要理解 AOP 的概念，我们先用 OOP 举例，比如一个业务组件 BookService，它有几个业务方法：

createBook：添加新的 Book；updateBook：修改 Book；deleteBook：删除 Book。

对每个业务方法，例如，createBook()，除了业务逻辑，还需要安全检查、日志记录和事务处理。

对于安全检查、日志、事务等代码，它们会重复出现在每个业务方法中。使用 OOP，我们很难将这些四处分散的代码模块化。

把日志、事务也视为切面，然后，以某种自动化的方式，把切面织入到核心逻辑中，实现 Proxy 模式。

第一步：引入 AOP 的依赖。

```
<!-- Spring AOP核心 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>6.2.7</version>
</dependency>
<!-- AspectJ支持 -->
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.9.1</version>
</dependency>
```

第二步：开启 AOP 注解支持。

```
@Configuration //标记类为 Spring 配置类, 替代 XML 的<beans>标签
@ComponentScan("com.edu.springedu") //配置容器自动扫描指定包下的组件
@EnableAspectJAutoProxy // 开启AOP代理
public class SpringConfig {
```

第三步：编写切面代码。

```
@Aspect // 标记为切面
@Component // 纳入容器管理
public class LogAspect {

    // 切点: 匹配UserService的所有方法
    @Before("execution(* com.edu.springedu.entity.Hello.*(..))") no usages
    public void beforeMethod() {
        System.out.println("AOP前置通知: 方法执行前记录日志");
    }

    @After("execution(* com.edu.springedu.entity.Hello.*(..))") no usages
    public void afterMethod() {
        System.out.println("AOP后置通知: 方法执行后记录日志");
    }
}
```

第四步：测试。

```
@Test
public void test03() {
    ApplicationContext ctx = new AnnotationConfigApplicationContext(SpringConfig.class);
    Hello hello = ctx.getBean(Hello.class);
    hello.sayHello();
}
```

🔍 :

✓ 1 test passed 1 test total, 854 ms

```
D:\programes\Java\JDK17\bin\java.exe ...
AOP前置通知: 方法执行前记录日志
Hello, Person{name='韩梅梅', age=18, address='中国深圳'}
AOP后置通知: 方法执行后记录日志
```

## SpringBoot3 框架

官网: <https://spring.io/>

Spring Boot 是一个基于 Spring 的套件, 它帮我们预组装了 Spring 的一系列组件, 以便以尽可能少的代码和配置来开发基于 Spring 的 Java 应用程序。

|             | Spring Boot 2.x  | Spring Boot 3.x    |
|-------------|------------------|--------------------|
| Spring版本    | Spring 5.x       | Spring 6.x         |
| JDK版本       | >= 1.8           | >= 17              |
| Tomcat版本    | 9.x              | 10.x               |
| Annotation包 | javax.annotation | jakarta.annotation |
| Servlet包    | javax.servlet    | jakarta.servlet    |
| JMS包        | javax.jms        | jakarta.jms        |
| JavaMail包   | javax.mail       | jakarta.mail       |

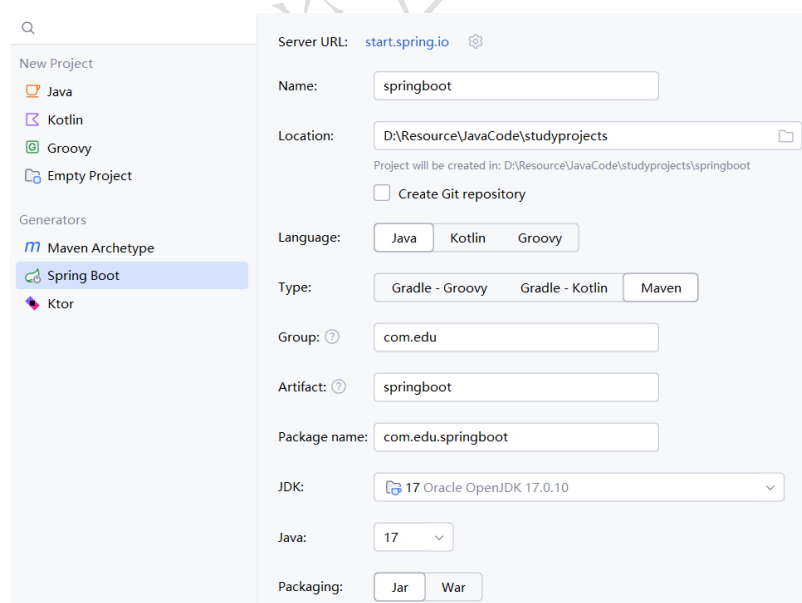
### ➤ 什么是 Spring Boot?

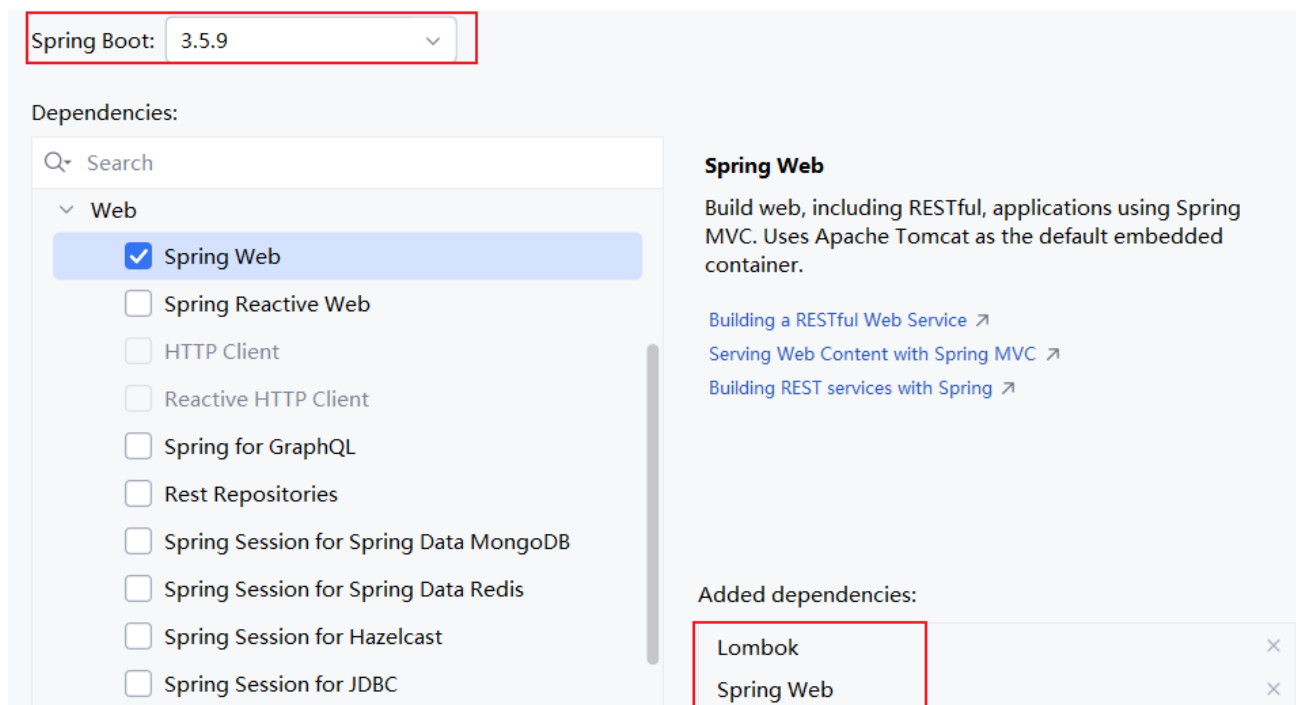
Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

能快速创建出生产级别的 Spring 应用。

## 入门案例

第一步: 使用 Idea 快速创建 Spring Boot 项目。





在这个页面选择 spring web 和 Lombok，点击 Create。这样 Idea 会帮我们创建一个 spring boot 工程。

第二步：编写业务 Controller 类。

`@RestController` //说明返回数据对象  
`public class HomeController {`

```

    @GetMapping("/hello")
    public String home() {
        return "Hello Spring Boot";
    }

```

第三步：测试运行。

localhost:8080/hello

综合

## Hello Spring Boot

也可以配置端口：

application.properties

```

1 spring.application.name=springboot01
2 server.port=8090

```

第四步：项目打包运行。

Maven → package

控制台到项目的 target 目录，运行 `Java -jar XXXXXX.jar`。

## Spring 与 SpringBoot

Spring 和 Spring Boot 基于 IOC AOP 理念实现，Spring Boot 集成了 Spring。

### ➤ Spring 框架

Spring 框架解决了企业级开发的复杂性，它是一个容器框架，用于装载 Java 对象(JavaBean)，使程序间的依赖关系交由容器统一管理，松耦合，提高了可测试性和维护效率。Spring 主要做了两件事，1：省去了程序员工手工创建对象的操作；2：声明了属性赋值。

### ➤ Spring Boot 框架

Spring Boot 是一个微服务框架，延续了 Spring 框架的核心思想 IOC 和 AOP，简化了应用的开发和部署。Spring 和 Spring Boot 的最大的区别在于 Spring Boot 的自动装配原理。

## 开发技巧

### ➤ Lombok 使用

Lombok 是一个 Java 库，能自动插入编辑器并构建工具，简化 Java 开发。通过添加注解的方式，不需要为类编写 getter 或 setter 方法，同时可以自动化日志变量。

简而言之：Lombok 能以简单的注解形式来简化 java 代码，提高开发人员的开发效率。

在 Idea 中安装插件 lombok（演示）。

```
@Data    //生成get set方法
@NoArgsConstructor    //生成无参构造函数
@AllArgsConstructor    //生成全参构造函数
@Component    //把当前类的实例放到spring boot容器中
public class Person {

    @Value("${person.name}")    //SPEL
    private String name;

    @Value("${person.address}")
    private String address;
}
```

对应 properties 文件

```
person.name=AdaWang
person.address=BeiJing
```

## ➤ 简化日志开发

```
/**
 * @RestController: 组合注解 = @Controller + @ResponseBody
 * 表示该类是控制器，且所有方法返回值直接转为JSON/字符串（而非视图）
 */

@RestController
@Slf4j //日志注解
@RequestMapping("/hello") // 统一请求路径前缀
public class HomeController {

    @Autowired
    private Person person;

    /**
     * @GetMapping: 处理GET请求，等价于@RequestMapping(method = RequestMethod.GET)
     * 访问路径: http://localhost:8080/hello/world
     */
    @GetMapping("/world")
    public Person hello() {
        log.info("Hello Method is Running...");
        return person;
    }
}
```

## SpringBoot 容器

想象一下，一个容器是一个可以容纳各种物品的地方，这些物品可以是食物、玩具或任何其他物品。同样地，Spring 容器负责容纳和管理应用程序中的对象。这些对象可以是 JavaBean、配置信息或其他类型的对象。

## ➤ 怎样在 spring boot 容器中获取对象

```
//初始化Spring Boot容器框架
ConfigurableApplicationContext ctx =
    SpringApplication.run(SpringdemoApplication.class, args);

System.out.println("=====");
for(String name : ctx.getBeanDefinitionNames()){
    System.out.println(name);
}
System.out.println("=====");
//获取容器中的对象
Person person = ctx.getBean(Person.class);
System.out.println(person);
```

## 再谈 IOC 与 DI

### ➤ IOC:

Inverse of Control 反转控制的概念。就是将原本在程序中手动创建对象的控制权，交由 Spring 框架管理，简单说，就是创建对象控制权被反转到了 spring 框架。

### ➤ DI:

Dependency Injection : 依赖注入，在 spring 框架负责创建 Bean 对象时，动态的将依赖对象注入到 Bean 组件中。

## Spring Boot 是怎样创建对象的？

### ➤ 第一种途径：通过配置类的@Bean 注解。

#### 1. @Configuration 注解

@Configuration 注解是 Spring 框架中常用的注解之一，它表示该类是一个配置类，用于定义 Spring 应用上下文中的 Bean。具体来说，@Configuration 注解表示将该类作为一个 Spring 配置类进行处理，其中定义的 Bean 对象会由 Spring 容器进行实例化和管理的。代替了以前那些 xml 配置文件。

#### 2. @Bean 注解

Spring 的@Bean 注解用于告诉方法，产生一个 Bean 对象，然后这个 Bean 对象交给 Spring 管理。产生这个 Bean 对象的方法 Spring 只会调用一次，随后这个 Spring 将会将这个 Bean 对象放在自己的 IOC 容器中。

```

@Configuration    //说明当前是配置类
public class SpringConfig {

    @Bean    //把当前方法返回的对象放到spring boot容器中
    public User getUser() { return new User( name: "admin", password: "123456"); }
}

```

@Bean 是一个方法级别上的注解，主要用在@Configuration 注解的类里，也可以用在@Component 注解的类里。添加的 bean 的 id 为方法名。

## ➤ 第二种途径：通过@Component 注解及其子注解。

### 1. @Component 注解

标注 Spring 管理的 Bean，使用@Component 注解在一个类上，表示将此类标记为 Spring 容器中的一个 Bean。

基于@Component 扩展的注解：

@Repository：是基于@Component 注解的扩展，被@Repository 注解的类表示 DAO 层实现。

@Service：是基于@Component 注解的扩展，被@Service 注解的类表示业务逻辑层实现。

@Controller：是基于@Component 注解的扩展，被@Controller 注解的类表示 Web 层实现。

而@RestController 是基于@Controller 注解的扩展。

## Spring Boot 是怎样依赖注入属性及对象的？

### ➤ 第一种：通过@Value 注解给对应的属性赋值

#### 1. @value 注解

此注解的功能是将与配置文件中的键对应的值分配给其带注解的属性。

```

@Data    //生成get set方法
@NoArgsConstructor    //生成无参构造函数
@AllArgsConstructor    //生成全参构造函数
@Component    //把当前类的实例放到spring boot容器中
public class Person {

    @Value("${person.name}")    //SPEL
    private String name;

    @Value("${person.address}")
    private String address;
}

```

```

application.properties
1  spring.application.name=springdemo
2
3  person.name=AdaWang
4  person.address=BeiJing

```

➤ 第二种：通过@Autowired 注解给对象注入对象属性。

## 2. @Autowired 注解

把容器中对应的对象依赖注入到当前类中。

```
@SpringBootTest    //初始化spring boot 容器
class SpringdemoApplicationTests {

    @Autowired    //把Spring Boot容器对应的person对象依赖注入到该类中
    private Person person;    //把Person对象当作测试类的属性

    @Test    //运行单元测试
    public void test01() {
        System.out.println(person);
    }
}
```

## Starters 启动器

Starters 可以理解为启动器，它包含了一系列可以集成到应用里面的依赖包，你可以一站式集成 Spring 及其他技术，而不需要到处找示例代码和依赖包。如你想使用 Spring JPA 访问数据库，只要加入 spring-boot-starter-data-jpa 启动器依赖就能使用了。

### ➤ Starters 命名

Spring Boot 官方的启动器都是以 spring-boot-starter-命名的，代表了一个特定的应用类型。

第三方的启动器不能以 spring-boot 开头命名，它们都被 Spring Boot 官方保留。一般第三方的应该这样命名，像 mybatis 的 mybatis-spring-boot-starter。

### ➤ Starters 分类

| 启动器名称                        | 功能描述                                               |
|------------------------------|----------------------------------------------------|
| spring-boot-starter          | 包含自动配置、日志、YAML 的支持。                                |
| spring-boot-starter-web      | 使用 Spring MVC 构建 web 工程，包含 restful，默认使用 Tomcat 容器。 |
| spring-boot-starter-actuator | 提供生产环境特性，能监控管理应用。                                  |

## ➤ 官方的 Starters

|                                                     |                                                                                                                           |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>spring-boot-starter-security</code>           | Starter for using Spring Security                                                                                         |
| <code>spring-boot-starter-test</code>               | Starter for testing Spring Boot applications with libraries including JUnit Jupiter, Hamcrest and Mockito                 |
| <code>spring-boot-starter-thymeleaf</code>          | Starter for building MVC web applications using Thymeleaf views                                                           |
| <code>spring-boot-starter-validation</code>         | Starter for using Java Bean Validation with Hibernate Validator                                                           |
| <code>spring-boot-starter-web</code>                | Starter for building web, including RESTful, applications using Spring MVC. Uses Tomcat as the default embedded container |
| <code>spring-boot-starter-data-elasticsearch</code> | Starter for using Elasticsearch search and analytics engine and Spring Data Elasticsearch                                 |
| <code>spring-boot-starter-data-jdbc</code>          | Starter for using Spring Data JDBC                                                                                        |
| <code>spring-boot-starter-data-jpa</code>           | Starter for using Spring Data JPA with Hibernate                                                                          |
| <code>spring-boot-starter-aop</code>                | Starter for aspect-oriented programming with Spring AOP and AspectJ                                                       |

这些 starters 几乎涵盖了 Java EE 所有常用场景，Spring Boot 对这些场景依赖的 jar 也做了严格的测试与版本控制。我们不必担心 jar 版本兼容性问题。

## SpringBoot3 启动原理

从核心入口、关键注解、核心流程三个维度展开，理解 “为何 Spring Boot 能一键启动”。

### ➤ 三个基础概念

- 1) Spring Boot 核心目标：简化 Spring 应用的配置、启动、部署，核心是「约定大于配置」+「自动配置」。
- 2) Spring Boot3 vs Spring6: Spring Boot3 基于 Spring 6，要求 JDK 17+，底层适配 Jakarta EE 9+（包名从 `javax.*` 改为 `jakarta.*`），但启动核心逻辑与 Spring Boot2 一致。
- 3) 核心入口：SpringApplication.run() 方法是启动的唯一入口，所有逻辑围绕该方法展开。

## ➤ 启动核心注解解析

启动类上的@SpringBootApplication 注解是启动的 “总开关”，先拆解这个注解的构成。

### 1) @SpringBootApplication 注解构成

```
// 简化后的注解源码（仅保留核心）
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
// 核心子注解1：标记为Spring配置类（替代XML）
@Configuration
// 核心子注解2：开启组件扫描（默认扫描当前类所在包及其子包）
@ComponentScan
// 核心子注解3：开启自动配置（Spring Boot的核心）
@EnableAutoConfiguration
public @interface SpringBootApplication {
    // 省略属性...
}
```

| 子注解                      | 核心作用                                                     |
|--------------------------|----------------------------------------------------------|
| @Configuration           | 将启动类标记为 Spring 的配置类，容器会加载该类中的 Bean 定义                    |
| @ComponentScan           | 自动扫描启动类所在包及其子包下的 @Component/@Service/@Controller 等组件     |
| @EnableAutoConfiguration | 最核心！触发 Spring Boot 的自动配置机制，自动加载默认配置（如 Tomcat、Spring MVC） |

### 2) @EnableAutoConfiguration 底层逻辑

@EnableAutoConfiguration 是自动配置的核心，它通过两个关键类实现：

（1）AutoConfigurationImportSelector：加载

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件（Spring Boot 3 新增，替代旧版的 spring.factories）。

（2）该文件中定义了所有自动配置类

如 TomcatAutoConfiguration、DispatcherServletAutoConfiguration，Spring Boot 会根据类路径中的依赖（如引入 spring-boot-starter-web 则加载 Tomcat+MVC 配置）自动启用对应配置。

## ➤ Spring Boot3 启动核心流程

### ✓ 阶段 1：初始化 SpringApplication 对象。

此阶段主要做“准备工作”——识别应用类型（如 Web 应用则加载 Servlet 相关配置）、加载扩展组件（初始化器 / 监听器）。

### ✓ 阶段 2：配置应用上下文（ApplicationContext）。

核心作用：上下文是 Spring 的 IoC 容器，负责管理 Bean 的创建、依赖注入。

### ✓ 阶段 3：加载自动配置类。

1. AutoConfigurationImportSelector。

2. 筛选出符合条件的自动配置类。

3. 将这些自动配置类注册到上下文，替代手动编写的 Spring 配置（如无需手动配置 DispatcherServlet）。

### ✓ 阶段 4：刷新应用上下文（核心阶段）

refresh() 方法是 Spring 容器的核心，Spring Boot 复用了 Spring 的容器刷新逻辑，完成：BeanFactory 初始化：创建 Bean 工厂，解析配置类中的 Bean 定义。

Bean 后置处理器注册：注册处理注解（如@Autowired）的处理器。

Bean 实例化：创建所有非懒加载的 Bean（如控制器、服务类）。

初始化 Web 服务器：如果是 Web 应用，启动内置 Tomcat/Jetty（核心!）。

### ✓ 阶段 5：启动内置服务器（Web 应用特有）

如果是 Web 应用，刷新上下文时会触发 ServletWebServerApplicationContext 的 onRefresh() 方法：

创建内置服务器（默认 Tomcat）：TomcatWebServer。

配置服务器端口、上下文路径等。

启动服务器，绑定端口（如 8080），监听 HTTP 请求。

### ✓ 阶段 6：启动完成，发布事件

发布 ApplicationStartedEvent 事件，通知监听器应用已启动。

输出启动日志。

应用进入运行状态，等待处理请求。

## 集成 MP

官网：<https://baomidou.com/>

MyBatis-Plus（简称 MP）是一个 MyBatis 的增强工具，在 MyBatis 的基础上只做增强不做改变，为简化开发、提高效率而生。

## 入门案例

开始入门案例之前，需要创建好一个数据库、表，准备好 JDK、Idea、Maven 等开发环境。

第 1 步：使用 Idea 快速创建一个 spring boot3 初始化项目。

第 2 步：手动配置 pom.xml（核心依赖），在项目的 pom.xml 中添加 MyBatis-Plus 依赖（Spring Boot 3 适配版）。

```
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
    <version>3.5.7</version>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
</dependency>
```

第 3 步：配置数据库和 MP。

```
spring.datasource.url=jdbc:mysql://localhost:3306/customer?useSSL=false&characterEncoding=utf8&serverTimezone=Asia/Shanghai
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=root

mybatis-plus.configuration.map-underscore-to-camel-case=true
mybatis-plus.configuration.log-impl=org.apache.ibatis.logging.stdout.StdoutImpl
mybatis-plus.type-aliases-package=com.alibaba.springmp.entity
mybatis-plus.mapper-locations=classpath:mapper/*.xml
```

第 4 步：创建实体类。

```

@Data 23 usages
@TableName("sys_user")    //指定表名
public class User {

    @TableId(type = IdType.AUTO)    //指定主键
    private Long id;

    private String username;

    private String password;

    private String nickName;    //nick_name

    private Integer status;
}

```

第 5 步：创建 Mapper 接口。MP 核心优势是无需编写 XML / 注解 SQL，只需继承 BaseMapper<T> 即可拥有 CRUD 功能。

```

public interface UserMapper extends BaseMapper<User> {

}

```

第 6 步：启动类(确保扫描到 Mapper)。修改项目启动类 Springboot3MpDemoApplication.java，可选添加 @MapperScan（替代每个 Mapper 加 @Mapper）。

```

@SpringBootApplication
@MapperScan("com.alibaba.springmp.mapper") //映射器扫描器
public class SpringmpApplication {

```

第 7 步：测试 MP 功能。

```

@SpringBootTest
class SpringmpApplicationTests {

    @Autowired
    private UserMapper userMapper;

    @Test
    public void test01() {
        User user = new User();
        user.setUsername("user");
        user.setPassword("123");
        user.setNickName("普通用户");
        user.setStatus(1);
        userMapper.insert(user);
    }
}

```

```

@Test
public void test02() {
    User user = new User();
    user.setId(3L);
    user.setUsername("tom");
    user.setPassword("tom");
    user.setNickName("商户");
    user.setStatus(1);
    userMapper.updateById(user);
}

@Test
public void test03() {
    User user = userMapper.selectById(1L);
    System.out.println(user);
}

@Test
public void test04() {
    List<User> users = userMapper.selectList(queryWrapper: null);
    for (User user : users) {
        System.out.println(user);
    }
}

@Test
public void test05() { userMapper.deleteById(3L); }

```

## MP 核心功能拓展

### ➤ 条件查询 (QueryWrapper):

```

QueryWrapper<User> queryWrapper = new QueryWrapper<>();
queryWrapper.eq(column: "username", val: "admin");
queryWrapper.orderByDesc(column: "id");
List<User> users = userMapper.selectList(queryWrapper);
users.stream().forEach(System.out::println);

```

其它查询可参考官方文档。

### ➤ 分页查询

```

@Configuration
public class MybatisPlusConfig {

    /**
     * 添加分页插件
     */
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL));
        // 如果有多数据源可以不配具体类型，否则都建议配上具体的 DbType
        return interceptor;
    }
}

```

测试分页：

```

@Test
public void testPage() {
    // 第 1 页，每页 2 条数据
    Page<User> page = new Page<>(current: 1, size: 2);
    userMapper.selectPage(page, queryWrapper: null);
    System.out.println("总记录数: " + page.getTotal());
    System.out.println("总页数: " + page.getPages());
    page.getRecords().forEach(System.out::println); // 打印当前页数据
}

```

### ➤ 主键策略

#### ✓ AUTO 自增策略

需要在创建数据表的时候设置主键自增

实体字段中配置 @TableId(type = IdType.AUTO)

```

@TableId(type = IdType.AUTO) //主键自增长 主键注解
private Integer customerId; // customer_id =>customerId

```

#### ✓ ASSIGN\_ID 分布式 ID

MyBatis-Plus 默认的主键策略是：ASSIGN\_ID（使用了雪花算法）。

```

@TableId(type = IdType.ASSIGN_ID) //分布ID 主键注解
private Long userId;

```

### ➤ 统一添加修改字段

```

@Component
public class MyMetaObjectHandler implements MetaObjectHandler {
    @Override 1 usage
    public void insertFill(MetaObject metaObject) {
        this.strictInsertFill(metaObject, fieldName: "createTime",
            LocalDateTime.class, LocalDateTime.now());
        this.strictInsertFill(metaObject, fieldName: "updateTime",
            LocalDateTime.class, LocalDateTime.now());
    }

    @Override 1 usage
    public void updateFill(MetaObject metaObject) {
        this.strictUpdateFill(metaObject, fieldName: "updateTime",
            LocalDateTime.class, LocalDateTime.now());
    }
}

```

## 代码生成器

全新的 MyBatis-Plus 代码生成器，通过 builder 模式可以快速生成你想要的代码，快速且优雅，跟随下面的代码一睹为快。

第一步：引入依赖。

```

<!-- 代码生成器依赖 -->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-generator</artifactId>
    <version>3.5.14</version>
</dependency>
<dependency>
    <groupId>org.freemarker</groupId>
    <artifactId>freemarker</artifactId>
    <version>2.3.28</version>
</dependency>

```

第二步：快速代码生成器。

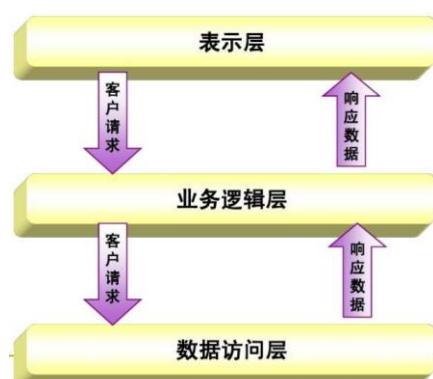
CodeGenerator.java

```
FastAutoGenerator.create("url", "username", "password")
    .globalConfig(builder -> builder
        .author("Baomidou")
        .outputDir(Paths.get(System.getProperty("user.dir")) + "/src/main/java")
        .commentDate("yyyy-MM-dd")
    )
    .packageConfig(builder -> builder
        .parent("com.baomidou.mybatisplus")
        .entity("entity")
        .mapper("mapper")
        .service("service")
        .serviceImpl("service.impl")
        .xml("mapper.xml")
    )
    .strategyConfig(builder -> builder
        .entityBuilder()
```

## 三层结构

三层架构是一种常见的软件设计模式，旨在提高代码的可维护性、可扩展性。它将系统分为表现层（UI）、业务逻辑层（BLL）和数据访问层（DAL）。UI 层负责用户交互，BLL 层处理业务逻辑，而 DAL 层则与数据库交互。三层架构通过实体类作为数据传递的载体，实现高内聚、低耦合。使用三层架构能解耦组件，提高代码的可维护性和适应需求变化的能力。相比于两层架构，三层架构的耦合度更低，可扩展性更强。

在 Web 开发中，三层架构是指：视图层 view（表现层），服务层 service（业务逻辑层），持久层 Dao（数据访问层）。





## 什么是三层？

**UI (view 表现层):** 主要是指与用户交互的界面。用于接收用户输入的数据和显示处理后用户需要的数据。

**BLL: (service 业务逻辑层):** UI 层和 DAL 层之间的桥梁。实现业务逻辑。业务逻辑具体包含：验证、计算、业务规则等等。

**DAL: (dao/mapper 数据访问层):** 与数据库打交道。主要实现对数据的增、删、改、查。将存储在数据库中的数据提交给业务层，同时将业务层处理的数据保存到数据库。

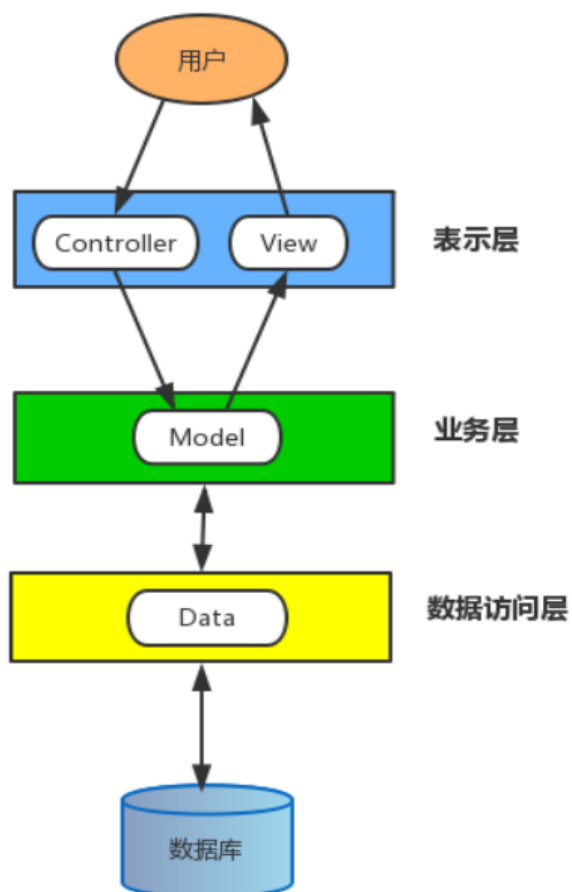
## MVC 设计模式

**MVC 设计模式:** Model-View-Controller 简写。MVC 是软件工程中的一种软件架构模式，它是一种分离业务逻辑与显示界面的设计方法，它把软件系统分为三个基本部分：模型 (Model)、视图 (View)、控制器 (Controller)。

- ✧ **控制器 Controller:** 对请求进行处理，负责请求转发，也就是事件，用于流程的控制；
- ✧ **视图 View:** 界面设计人员进行图形界面设计，也就是 GUI 组件，用于数据的展示；
- ✧ **模型 Model:** 编写程序应用功能（实现算法等等），数据库管理，也就是实体类。用于数据的封装和数据的传输；

### ➤ 三层架构与 MVC 的关系

MVC 是在三层架构的基础上设计的一种框架型架构，三层架构是一种宏观的概念，而 MVC 就是一种比较具体的三层架构的框架实现。



## Spring Web 开发

Spring Boot 的 Web 开发是通过 Spring MVC 实现的。对于 web 开发有二个方向：

一个是：前后端分离的开发模式，即提交请求返回数据对象；

一个是：网页的开发模式，即提交请求返回页面对象。这里的页面对象，Spring 官方推荐的是 thymeleaf 模板页面。

### ➤ 资源目录

静态资源目录/static : 这里就用来存放静态资源文件的。比如 js、css 文件、图片等。

模板页面目录/ templates : 这里是用来进行 thymeleaf 页面开发存放页面的。

## 什么是 spring mvc?

Spring MVC 是 Spring 框架的核心模块之一,是基于 MVC 设计模式(Model-View-Controller) 的 Web 开发框架,主要用于开发 Java Web 应用。Spring Boot 集成了 spring MVC。

Model (模型): 处理数据 (实体类、Service、Mapper 等);

View (视图): 展示数据 (JSP/Thymeleaf/HTML , JSON 数据等);

Controller (控制器): 接收请求、处理逻辑、返回响应。

### ➤ Spring MVC 的工作流程

第一步: 客户端发送请求到 Spring MVC 的控制层,通过前端控制器 (DispatcherServlet) 查询 Controller 层的 URL, 并找到对应的 URL 执行对应的控制层的方法, 如果没找到则报 404 错误。

第二步: 执行控制层对应方法, 完成业务逻辑。

第三步: 返回正确的数据对象、页面对象或执行异常业务逻辑的错误信息。

## 返回视图对象

Spring MVC 返回视图对象是通过@Controller 注解实现的。

Thymeleaf 是一个服务器端 Java 模板引擎,能够处理 HTML、XML、CSS、JAVASCRIPT 等模板文件。Thymeleaf 模板可以直接当作静态原型来使用,它主要目标是为开发者的开发工作流程带来优雅的自然模板,也是 Java 服务器端 HTML5 开发的理想选择。Spring Boot 官方默认使用 thymeleaf 作为页面渲染组件。

参考网址:

<https://www.wuzao.com/thymeleaf/doc/articles/standarddialect5minutes.html>

第 1 步: 在 pom.xml 中导入 thymeleaf 启动器。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

注意: spring boot 关于 thymeleaf 自动配置的源代码如下。

```
public class ThymeleafProperties {
    private static final Charset DEFAULT_ENCODING;
    public static final String DEFAULT_PREFIX = "classpath:/templates/";
    public static final String DEFAULT_SUFFIX = ".html";
    private boolean checkTemplate = true;
    private boolean checkTemplateLocation = true;
    private String prefix = "classpath:/templates/";
    private String suffix = ".html";
    private String mode = "HTML";
}
```

从源代码中可知，模板页面是放在 spring boot 工程 resources 目录下的 templates 目录下的，并以 html 为扩展名。

第 2 步：写一个 Controller 类，在写一个方法，返回页面对象。

```
@GetMapping("/hello")
public String index(){
    return "success";
}
```

第 3 步：在 templates 目录下添加 success.html 页面，并告诉 spring boot，我是 thymeleaf 模板页面。

```
<> success.html x
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <meta charset="UTF-8">
5     <title>Thymeleaf模板页面</title>
6 </head>
```

第 4 步：在 html 页面就可以使用 thymeleaf 语法开发动态页面了。

## Thymeleaf 核心语法

### ➤ Thymeleaf 常用表达式

Thymeleaf 的核心是表达式，用于从 Model 中获取数据、处理逻辑，以下是最常用的类型：

| 表达式类型   | 语法                | 作用                      |
|---------|-------------------|-------------------------|
| 变量表达式   | <code>\${}</code> | 获取 Model 中的数据 (核心)      |
| 选择变量表达式 | <code>*{}</code>  | 基于当前对象获取属性 (简化代码)       |
| 消息表达式   | <code>#{}</code>  | 读取国际化配置文件 (多语言)         |
| 链接表达式   | <code>@{}</code>  | 生成 URL (路径、参数拼接)        |
| 片段表达式   | <code>~{}</code>  | 复用 HTML 片段 (如公共页眉 / 页脚) |

```
<!DOCTYPE html>
<!-- 必须声明 Thymeleaf 命名空间, 否则 th:* 属性无效 -->
<html lang="zh-CN" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="${title}">默认标题</title> <!-- 动态设置标题 -->
</head>
<body>
    <h1>1. 变量表达式 ${}: 获取单个对象属性</h1>
    <div>
        ID: <span th:text="${user.id}">1</span><br>
        姓名: <span th:text="${user.name}">张三</span><br>
        年龄: <span th:text="${user.age}">18</span><br>
        邮箱: <span th:text="${user.email}">zhangsan@test.com</span>
    </div>
```

```
<hr>
<h1>2. 选择变量表达式 *{}: 简化对象属性获取</h1>
<div th:object="${user}"> <!-- 指定当前对象为 user -->
    ID: <span th:text="*{id}">1</span><br>
    姓名: <span th:text="*{name}">张三</span><br>
    年龄: <span th:text="*{age}">18</span>
</div>
```

```

<!-- th:each="遍历项, 状态变量 : 数据源" -->
<tr th:each="u, stat : ${userList}">
    <td th:text="${u.id}">1</td>
    <td th:text="${u.name}">张三</td>
    <td th:text="${u.age}">18</td>
    <td th:text="${u.email}">zhangsan@test.com</td>
    <td th:text="${stat.index}">索引（从0开始）</td>
    <td th:text="${stat.count}">计数（从1开始）</td>
</tr>

```

```

<!-- 遍历 Map -->
<div th:each="entry : ${cityMap}">
    区号: <span th:text="${entry.key}">010</span> → 城市: <span th:text="${entry.value}">北京</span>
</div>

<hr>
<h1>4. 条件判断 th:if/th:unless/th:switch</h1>
<!-- th:if: 条件为 true 时显示 -->
<div th:if="${user.age >= 18}">
    <p th:text="${user.name} + '是成年人'">张三是成年人</p>
</div>

```

```

<!-- th:switch: 多条件分支 -->
<div th:switch="${user.age}">
    <p th:case="18">年龄是18</p>
    <p th:case="20">年龄是20</p>
    <p th:case="*">其他年龄</p> <!-- * 表示默认 -->
</div>

```

```

<h1>5. 链接表达式 @{}: 生成 URL</h1>
<!-- 普通链接 -->
<a th:href="@{/thymeleaf/demo}">普通链接</a><br>
<!-- 带参数的链接 -->
<a th:href="@{/user/getById(id=${user.id}, name=${user.name})}">带参数的链接</a><br>
<!-- 静态资源（CSS/JS/图片） -->
 <!-- 实际路径: static/images/logo.png -->

```

```

<h1>6. 字符串拼接/运算</h1>
<!-- 方式1: + 拼接 -->
<p th:text="姓名: ${user.name} + ', 年龄: ' + ${user.age}">姓名: 张三, 年龄: 18</p>
<!-- 方式2: |{}| 简化拼接（推荐） -->
<p th:text="|姓名: ${user.name}, 年龄: ${user.age}|">姓名: 张三, 年龄: 18</p>
<!-- 数值运算 -->
<p th:text="${user.age + 1}">19</p> <!-- 年龄+1 -->
<!-- 逻辑运算 -->
<p th:text="${user.age >= 18 and user.name == '张三'}">true</p>

```

```
<h1>7. 内置对象</h1>
<!-- Web 内置对象: #request/#session 等 -->
<p>请求URL: <span th:text="${#request.getRequestURL()}">URL</span></p>
<p>SessionID: <span th:text="${#session.getId()}">SessionID</span></p>
<!-- 工具类对象: #strings/#numbers 等 -->
<p>姓名大写: <span th:text="${#strings.toUpperCase(user.name)}">ZHANGSAN</span></p>
<p>年龄格式化: <span th:text="${#numbers.formatInteger(user.age, 2)}">18</span></p> <!-- 补零到2位 -->
```

```
<h1>8. 片段复用 th:insert/th:replace</h1>
<!-- 引入公共片段（比如页眉），假设 templates/common/header.html 是公共片段 -->
<div th:insert="~{common/header :: header}"></div>
<!-- 简化写法（省略 ~{ }） -->
<div th:insert="common/header :: header"></div>
```

➤ 常用 Thymeleaf 属性

| 属性       | 作用                       | 示例                                            |
|----------|--------------------------|-----------------------------------------------|
| th:text  | 替换标签文本（转义 HTML）          | <p th:text="\${user.name}">默认值</p>            |
| th:utext | 替换标签文本（不转义 HTML）         | <p th:utext="\${htmlContent}">默认</p>          |
| th:value | 设置 input 标签的 value 属性    | <input th:value="\${user.name}">              |
| th:src   | 设置 img/script 标签的 src 属性 |             |
| th:href  | 设置 a 标签的 href 属性         | <a th:href="@{/user/list}">列表</a>             |
| th:class | 设置标签的 class 属性           | <div th:class="\${isActive ? 'active' : ''}"> |
| th:style | 设置标签的 style 属性           | <div th:style="'color: red;'">                |

➤ 公共片段复用

Thymeleaf 最实用的功能之一是复用公共 HTML 片段（比如页眉、页脚、导航栏），步骤如下：

1. 创建公共片段文件

在 templates/common/header.html 中编写公共页眉：

```

<!DOCTYPE html>
<html lang="zh-CN" xmlns:th="http://www.thymeleaf.org">
<!-- 定义片段: name="header" -->
<div th:fragment="header">
    <div style="background: #eee; padding: 10px;">
        <h2>这是公共页眉</h2>
        <a th:href="@{/thymeleaf/demo}">返回演示页</a>
    </div>
</div>
</html>

```

## 2. 引入公共片段

在需要的模板中通过 `th:insert/th:replace` 引入：

`th:insert`：保留自身标签，将片段插入到当前标签内；

`th:replace`：替换当前标签为片段标签（推荐，结构更简洁）。

```

<!-- insert: 当前 div 标签保留，片段插入其中 -->
<div th:insert="common/header :: header"></div>

<!-- replace: 当前 div 标签被片段的 div 替换 -->
<div th:replace="common/header :: header"></div>

```

### ➤ Thymeleaf 总结

核心基础：Thymeleaf 依赖 `th:*` 属性和 `${}/@{}` 等表达式，需先声明命名空间 `xmlns:th="http://www.thymeleaf.org"`；

高频用法：`${}` 取数据、`th:each` 遍历列表、`@{}` 生成 URL、`th:if` 条件判断、片段复用简化页面开发；

关键优势：模板文件可直接预览，与 Spring Boot 无缝集成，替代 JSP 成为主流。

## 返回数据对象

Spring MVC 返回数据对象是通过@RestController 注解实现的，此注解是@Controller 注解和@ResponseBody 注解作用之和。

Spring ResponseEntity 是一个类，它表示整个 HTTP 响应，包括状态码，标头和正文。我们可以使用它来完全配置 HTTP 响应。

ResponseEntity 继承了 HttpEntity 类，HttpEntity 代表一个 HTTP 请求或者响应实体，其内部有两个成员变量：header 和 body，代表 HTTP 请求或响应的 header 和 body，其中的 body 是泛型的。

定义统一响应结果集类，如下：

```
@Data 46 usages
public class R<T> {

    private Integer code; //编码: 200成功, 500 错误信息

    private String msg; //成功/错误信息

    private T data; //结果集

    private R() { no usages
    }

    private R(Integer code, String msg, T data) { 4 usages
        this.code = code;
        this.msg = msg;
        this.data = data;
    }

    //正确的响应信息，并返回结果集
    public static <T> R<T> success(String msg, T object) { return new R<>(code: 200, msg, object); }

    //正确的响应信息
    public static <T> R<T> success(String msg) { 4 usages
        return new R<>(code: 200, msg, data: null);
    }

    //失败的响应信息
    public static <T> R<T> failure(String msg) { return new R<>(code: 500, msg, data: null); }

    //自定义错误响应
    public static <T> R<T> error(int code, String msg) { return new R<>(code, msg, data: null); }
}
```

## RESTful API

### ➤ 什么是 RESTful?

REST (Representational State Transfer, 表述性状态转移) 是一种软件架构风格 (不是标准 / 协议), 核心是「资源」为中心, 通过 HTTP 方法表达对资源的操作, 实现无状态、可扩展的 API 设计。

**核心思想:** 把一切数据 / 服务都视为「资源」(比如用户、订单、商品), 用 URL 标识资源, 用 HTTP 方法定义操作。

### ➤ 核心特性:

**无状态:** 每个请求都包含所有必要信息, 服务器不存储客户端状态;

**统一接口:** 用标准 HTTP 方法 (GET/POST/PUT/DELETE) 操作资源;

**资源导向:** URL 只描述资源, 不包含操作 (比如 /users 而非 /getUser)。

### ➤ HTTP 方法与资源操作的对应关系

GET (SELECT): 从服务器取出资源 (一项或多项)。

POST (CREATE): 在服务器新建一个资源。

PUT (UPDATE): 在服务器更新资源 (客户端提供完整资源数据)。

DELETE (DELETE): 从服务器删除资源。

### ➤ RESTful API 设计规范 (必遵守)

规范是 RESTful 的灵魂, 遵循以下规则能让 API 清晰、易维护:

1. URL 设计: 资源导向, 只描述「是什么」

用名词复数表示资源集合 (比如 /users 而非 /user), 单个资源用 ID 区分 (/users/1);

URL 中只包含资源, 不包含操作 (禁止 /getUserById/1、/deleteUser/1 这种写法);

层级清晰, 体现资源关联 (比如 /users/1/orders 表示用户 1 的所有订单);

全小写, 用 - 分隔多单词 (比如 /user-addresses 而非 /userAddresses);

禁止使用动词 (比如 /createUser、/updateUser 都是错误的)。

2. 请求参数: 区分路径参数 / 查询参数

路径参数: 标识单个资源 (比如 /users/{id} 中的 id);

查询参数: 过滤、分页、排序 (比如 /users?page=1&size=10&sort=age, desc)。

### 3. 响应规范：统一格式 + 正确状态码。比如前面的 R 类。

```

@RestController
public class EmployeeController {

    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/emps/{page}")
    public ResponseEntity<Result<IPage<Employee>>> pageEmployee(
        @PathVariable("page") int page,
        @RequestParam(value = "name", required = false, defaultValue = "") String name){
        IPage<Employee> pe = employeeService.pageEmployee(page, name);
        return ResponseEntity.ok(Result.ok(message: "查询成功", pe));
    }

    @PostMapping("/emp")
    public ResponseEntity addEmployee(Employee employee){
        employeeService.addEmployee(employee);
        return ResponseEntity.ok(Result.ok("添加员工成功"));
    }

    @PutMapping("/emp")
    public ResponseEntity updateEmployee(@RequestBody Employee employee){
        employeeService.updateEmployee(employee);
        return ResponseEntity.ok(Result.ok("修改员工成功"));
    }

    @DeleteMapping("/emp/{empId}")
    public ResponseEntity deleteEmployee(@PathVariable("empId") Integer empId){
        employeeService.deleteEmployee(empId);
        return ResponseEntity.ok(Result.ok("删除员工成功"));
    }
}

```

### 异常统一处理

创建 common/GlobalExceptionHandler.java，捕获全局异常，避免接口返回非统一格式：

```

//全局异常处理
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class) //捕获ApiException异常 no usages
    public R exceptionHandler(ApiException e) { return R.failure(e.getMessage()); }
}

```

```
public class ApiException extends RuntimeException{ 7 usages

    public ApiException(String message) { super(message); }

}
```

## 参数校验

引入 spring-boot-starter-validation 依赖，对请求参数做校验：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

在实体类加校验注解：

```
@Data
public class User {
    private Long id;

    @NotBlank(message = "姓名不能为空") // 非空校验
    private String name;

    @Min(value = 1, message = "年龄不能小于1") // 最小值校验
    private Integer age;

    @Email(message = "邮箱格式错误") // 邮箱格式校验
    private String email;
}
```

在 Controller 中加 @Valid 触发校验：

```
@PostMapping
public Result<User> createUser(@Valid @RequestBody User user) {
    // ... 逻辑不变
}
```

在捕获全局异常类中加上以下代码，返回异常信息。

```
// 处理参数验证异常
@ExceptionHandler(MethodArgumentNotValidException.class) no usages
public R<Map<String, String>> handleValidationExceptions(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getAllErrors().forEach((ObjectError error) -> {
        String fieldName = ((FieldError) error).getField();
        String errorMessage = error.getDefaultMessage();
        errors.put(fieldName, errorMessage);
    });
    return R.error(code: 401, msg: "请求参数异常", errors);
}
```

## 拦截器

拦截器（Interceptor）是 Spring MVC 提供的核心组件，用于在请求处理的前后执行自定义逻辑，比如登录校验、日志记录、接口耗时统计、权限控制等。

它和过滤器（Filter）的区别：

拦截器是 Spring 层面的，基于反射实现，只能拦截 Controller 的请求；过滤器是 Servlet 层面的，基于函数回调，能拦截所有请求（包括静态资源）。

### ➤ 拦截器的核心方法：

preHandle：请求处理前执行（返回 true 放行，false 拦截）。

postHandle：请求处理后、视图渲染前执行。

afterCompletion：视图渲染完成后执行（无论是否异常都会执行，适合资源清理）。

### ➤ Spring Boot 实现拦截器的完整步骤

步骤 1：创建 Spring Boot 项目并引入依赖

确保 pom.xml（Maven）或 build.gradle（Gradle）中引入 Web 依赖：

```
<!-- Maven 依赖 -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

步骤 2：自定义拦截器（实现 HandlerInterceptor 接口）

创建一个拦截器类，实现 HandlerInterceptor 接口，重写核心方法。下面以「登录校验 + 接口耗时统计」为例：

```

/**
 * 自定义登录拦截器 + 接口耗时统计
 */
public class LoginInterceptor implements HandlerInterceptor { 1 usage
    private static final Logger log = LoggerFactory.getLogger(LoginInterceptor.class);
    // 记录请求开始时间
    private ThreadLocal<Long> startTime = new ThreadLocal<>(); 3 usages

    /**
     * 请求处理后、视图渲染前执行
     */
    @Override no usages
    public void postHandle(HttpServletRequest request, HttpServletResponse response,
        Object handler, ModelAndView modelAndView) throws Exception {
        log.info("请求处理完成, 准备渲染视图");
    }

    /**
     * 请求处理前执行 (核心: 登录校验)
     */
    @Override no usages
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
        Object handler) throws Exception {
        // 1. 记录请求开始时间
        startTime.set(System.currentTimeMillis());
        // 2. 获取请求路径
        String requestURI = request.getRequestURI();
        log.info("开始处理请求: {}", requestURI);
        // 3. 登录校验逻辑 (示例: 判断session中是否有user)
        Object user = request.getSession().getAttribute("user");
        if (user == null) {
            // 未登录: 返回401或重定向到登录页
            response.setStatus(401);
            response.getWriter().write("未登录, 请先登录!");
            return false; // 拦截请求
        }
        // 登录成功: 放行
        return true;
    }
}

```

```

/**
 * 视图渲染完成后执行（资源清理、耗时统计）
 */
@Override no usages
public void afterCompletion(HttpServletRequest request, HttpServletResponse response,
                           Object handler, Exception ex) throws Exception {
    // 1. 计算接口耗时
    long costTime = System.currentTimeMillis() - startTime.get();
    // 2. 清理ThreadLocal, 避免内存泄漏
    startTime.remove();
    // 3. 打印日志
    log.info("请求 {} 处理完成, 耗时: {}ms, 异常: {}",
            request.getRequestURI(), costTime, ex == null ? "无" : ex.getMessage());
}
}

```

步骤 3: 注册拦截器（实现 WebMvcConfigurer 接口）

Spring Boot 中需要通过配置类将拦截器注册到容器，并指定拦截 / 放行规则：

```

/**
 * Web MVC 配置类: 注册拦截器
 */
@Configuration // 标记为配置类, 让Spring扫描到
public class WebMvcConfig implements WebMvcConfigurer {

    /**
     * 注册拦截器, 并配置拦截规则
     */
    @Override no usages
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new LoginInterceptor()) // 添加自定义拦截器
            .addPathPatterns("/**") // 拦截所有请求
            .excludePathPatterns( // 放行的请求（登录、静态资源等）
                "/login", // 登录接口
                "/register", // 注册接口
                "/static/**", // 静态资源（css/js/img）
                "/error" // 错误页面
            );
    }
}

```

## 附：MySQL5.6 安装

### ➤ 步骤 1：下载安装包

访问 MySQL 官方下载地址：<https://downloads.mysql.com/archives/community/>

选择版本：

Product Version: 选择 5.6.x (如 5.6.51)

Operating System: 选择 Microsoft Windows

OS Version: 选择 Windows (x86, 64-bit), ZIP Archive (免安装版, 更灵活)

下载后解压到自定义目录 (如 D:\mysql-5.6.51-winx64), 建议路径不要有中文和空格。

### ➤ 步骤 2：配置环境变量

右键「此电脑」→「属性」→「高级系统设置」→「环境变量」；

在「系统变量」中找到 Path, 点击「编辑」；

添加 MySQL 的 bin 目录路径 (如 D:\mysql-5.6.51-winx64\bin)；

点击「确定」保存, 重启命令行生效。

### ➤ 步骤 3：初始化配置文件

在 MySQL 解压根目录下新建 my.ini 文件, 写入以下配置 (根据自己的路径修改):

```
[mysqld]
```

```
# 设置 3306 端口
```

```
port=3306
```

```
# 设置 mysql 的安装目录
```

```
basedir=D:\mysql-5.6.51-winx64
```

```
# 设置 mysql 数据库的数据的存放目录
```

```
datadir=D:\mysql-5.6.51-winx64\data
```

```
# 允许最大连接数
```

```
max_connections=200
```

```
# 字符集设置
```

```
character-set-server=utf8
```

```
# 数据库默认存储引擎
```

```
default-storage-engine=INNODB
```

```
[mysql]
```

```
# 设置 mysql 客户端默认字符集
```

```
default-character-set=utf8
```

#### ➤ 步骤 4：安装并启动 MySQL 服务

1. 以管理员身份打开 CMD 命令行，切换到 MySQL 的 bin 目录：

2. 初始化 MySQL（生成 data 目录和临时密码，MySQL 5.6 部分版本无需临时密码）：

```
mysqld --initialize-insecure # 无密码初始化（推荐新手）
```

```
# 或 mysqld --initialize # 生成随机临时密码，会输出在控制台
```

3. 安装 MySQL 服务（服务名默认是 MySQL，可自定义，如 MySQL56）：

```
mysqld -install MySQL56
```

4. 启动服务：

```
net start MySQL56
```

#### ➤ 步骤 5：登录并修改密码

1. 登录 MySQL（无密码初始化直接回车）：

```
mysql -u root -p
```

2. 修改 root 密码（替换为自己的密码，如 123456）：

— MySQL 5.6 修改密码语句

```
SET PASSWORD FOR 'root'@'localhost' = PASSWORD('123456');
```

— 刷新权限

```
FLUSH PRIVILEGES;
```

3. 退出：exit;

#### ➤ 常用的客户端工具

Navicate、SQLYog、HeidiSQL(推荐，下载见网址)。

<https://www.heidisql.com/download.php?download=installer>

#### ➤ MySQL 的卸载

第 1 步：停止 MySQL 服务。

方式 1：在系统服务中停止（services.msc）

方式 2：用 windows 命令行停止(net stop mysql)

第 2 步：卸载 。

mysqld remove

注意：安装过程中可能会出现“发生系统错误 2”。解决此问题，见：

<https://blog.csdn.net/njzgIdba/article/details/123564176>

注册表：regedit

